

Overview

This page summarizes practical research directions for **bus ETA prediction** and translates them into choices for TransitSense.

The main pattern across the literature is clear:

1. **Good ETA systems start with reliable route alignment and segment timing**, not with fancy models alone.
2. **Historical averages are simple but weak during traffic shifts and peak-hour variation.**
3. **Statistical and ML models improve accuracy when they use segment travel time, dwell time, time-of-day, and route context.**
4. **Deep models help most when there is enough historical data and a stable data pipeline.**
5. For a student MVP, the best choice is usually a **strong GPS + route-segment baseline first**, then a deeper model as a stretch layer.

Core sources reviewed

1. BusTr: Predicting Bus Travel Times from Real-Time Traffic
2. Bus Travel Time Prediction: A Lognormal Auto-Regressive (AR) Modeling Approach
3. Predicting Bus Travel Time with Hybrid Incomplete Data – A Deep Learning Approach
4. Bus Arrival Time Prediction via Hybrid LSTM Using GPS-Derived Run and Dwell Times
5. How Chalo Achieved 75% Bus ETA Prediction Accuracy Across 20+ Cities with AI

What the literature consistently says

1. The problem is not just prediction — it is pipeline quality

Most ETA errors come from upstream issues:

- noisy GPS points,
- low sampling rates,
- poor route alignment,
- stop dwell-time variability,
- and missing data during network gaps.

This means TransitSense should treat ETA as a **system design problem**:

- capture location reliably,
- snap the bus to the correct route segment,
- estimate per-segment travel time,
- adjust using current speed and recent segment conditions,
- then present ETA with a freshness/confidence signal.

2. Useful features repeat across papers

Commonly useful inputs include:

- current GPS position,
- bus speed,
- route ID / trip ID,
- stop sequence,
- segment-level historical travel time,
- dwell time at stops,
- hour of day,
- day of week,
- traffic condition proxies,
- and previous segment delays.

The strongest recurring insight is that **run time + dwell time + time-of-day context** are often enough to beat naive baselines.

3. Data richness determines model choice

- If only GPS is available, **statistical / simple ML approaches** are often the safest starting point.
- If route history and stop-level timing are available, **ANN/LSTM/hybrid models** become more useful.
- If external traffic feeds exist, **traffic-informed neural models** can improve performance further.

Method families

A. Historical / segment-average baseline

Problem

Predict bus arrival using route geometry plus average time per route segment or stop-to-stop section.

Data

- GPS location
- route shape / ordered stops
- historical travel time by segment
- optional day/time buckets

Method

- Snap current bus location to the nearest valid route segment.
- Compute remaining segments to a target stop.
- Sum expected segment times using recent or bucketed historical averages.
- Add a simple dwell-time allowance for important stops.

Results in practice

This is rarely the most accurate method, but it is often the **best starting benchmark** because it is interpretable

and easy to debug.

Constraints

- Weak during unusual congestion or incident days.
- Sensitive to route-alignment errors.
- Needs enough trip history to build stable averages.

Privacy stance

Strong. Uses vehicle telemetry only; no personal data required.

Hardware cost level

Low. A smartphone or low-cost GPS source is enough.

Connectivity assumptions

Can run with intermittent connectivity if the bus device buffers positions locally and uploads later.

Fit for TransitSense

Very strong as MVP baseline. It is simple, demoable, and gives a solid foundation for later ML improvements.

B. Statistical temporal models (AR / Kalman-style)

Problem

Improve ETA by modeling how delays evolve over time instead of treating each segment independently.

Data

- GPS-derived segment times
- recent historical delay values
- optionally route sections and time buckets

Method

The Lognormal AR paper models travel time as a temporally correlated process and exploits the lognormal behavior of travel times. Related work also uses Kalman-style updating to continuously revise predictions as new observations arrive.

Results

These methods generally improve over static averages, especially when traffic conditions evolve smoothly.

Constraints

- Better than simple averages, but still limited in capturing complex nonlinear patterns.
- Needs careful route segmentation and stable preprocessing.
- More mathematical tuning than the baseline.

Privacy stance

Strong. Still based mostly on vehicle telemetry.

Hardware cost level

Low.

Connectivity assumptions

Good for poor-network settings because updates can be computed centrally or from compact data streams.

Fit for TransitSense

Good candidate for a refined MVP or a strong baseline-plus model. More realistic than jumping directly to

heavy deep learning.

C. ANN / LSTM / hybrid sequence models

Problem

Capture nonlinear travel-time behavior caused by stop dwell variability, peak-hour dynamics, and historical pattern shifts.

Data

- GPS-derived run times
- dwell times
- stop IDs / sequence
- time-of-day, day-of-week
- route/trip context
- sometimes ticketing / smart-card signals

Method

The Hybrid LSTM paper combines sequential features such as stop IDs, run times, and dwell times with contextual inputs. The hybrid incomplete-data paper uses LSTM with incomplete data handling and additional optimization.

Results

The reviewed LSTM approach reported strong errors (MAE about 13.4 seconds, MAPE about 10.32%, RMSE about 24.26 seconds in its study setting), showing that even a relatively simple sequence model can outperform weaker baselines when the data pipeline is good.

Constraints

- Needs more historical data and cleaner preprocessing.
- Harder to explain to judges unless the pipeline is clear.
- Can silently fail if route alignment or dwell-time extraction is noisy.
- Hybrid models that require smart-card or richer feeds may not match student demo constraints.

Privacy stance

Still strong if only bus telemetry is used. Risk increases if passenger-linked data sources are introduced; avoid storing any identifiable user data.

Hardware cost level

Low to medium. Training is the heavier step; inference itself can still be lightweight.

Connectivity assumptions

Inference can happen server-side after periodic uploads. Not ideal for fully bus-side offline inference in a first MVP, but fine for a demo backend.

Fit for TransitSense

Best as an innovation layer or stretch goal. Good for the pitch, but should sit on top of a simpler operational baseline.

D. Traffic-informed neural models

Problem

Predict bus delay even where direct transit telemetry is limited, by learning from road traffic patterns.

Data

- road traffic forecasts
- historical bus delay data
- route mapping to roadway context

Method

BusTr learns to translate traffic conditions into bus travel-time predictions.

Results

The paper reports meaningful improvement over a strong neural baseline and shows practical value at large scale.

Constraints

- Depends on access to traffic data or a proxy for traffic conditions.
- Harder to reproduce in a hackathon unless traffic APIs are already available.
- More infrastructure-heavy than a pure GPS pipeline.

Privacy stance

Good. It does not require passenger identity data.

Hardware cost level

Medium on the system side because of data integration complexity.

Connectivity assumptions

More network-dependent because external traffic data usually needs online access.

Fit for TransitSense

Strong future enhancement, weak MVP fit. Excellent inspiration, but too dependent on external feeds for a first demo.

Comparison table

Design principles for TransitSense

Recommended baseline

Choose a **segment-based ETA engine** with:

1. route alignment,
2. per-segment historical travel times,
3. time-of-day buckets,
4. recent speed adjustment,
5. optional dwell-time penalty for major stops.

This gives the best balance of:

- feasibility,
- explainability,
- low hardware cost,
- and poor-network tolerance.

Recommended advanced option

If time permits, add a **small hybrid LSTM or ANN model** as a comparison layer for the pitch:

- baseline ETA vs ML ETA,
- same inputs plus dwell-time history and time context,
- show that the platform can evolve from heuristic to intelligent prediction.

Why this recommendation fits SIH

- It is **student-buildable**.
- It avoids over-dependence on external traffic APIs.
- It works with **smartphone-grade GPS or low-cost GPS hardware**.
- It remains **privacy-friendly** because it needs only bus movement data.
- It supports **offline-first buffering** when the network drops.

Constraints and risks to be explicit about

1. **Route alignment matters** — poor snapping can ruin ETA even with a good model.
2. **Data volume matters** — deep learning looks impressive, but weak history makes it unstable.
3. **Dwell-time variability matters** — school zones, major stops, and peak hours need explicit handling.
4. **Prediction freshness matters** — always show when the last bus update was received.
5. **Confidence matters** — ETA should be presented as more/less reliable depending on update recency and route certainty.

TransitSense recommendation

Default choice

Build the MVP around a **GPS + route-segment + historical travel-time ETA pipeline**.

Add if time permits

Add **AR/Kalman-style correction** or a **small LSTM comparison model**.

Avoid for first MVP

Do not depend on:

- smart-card integration,
- expensive sensors,
- or heavy online traffic-feed dependence.

These are useful in the literature, but they are not the best first move for a fast SIH prototype.